



UNIVERSIDAD POLITÉCNICA SALESIANA

CARRERA DE INGENIERÍA AUTOMOTRIZ

**Diseño e Implementación de una Interfaz para la Visualización y Registro de Datos
de Sensores Automotrices Simulados con Arduino y LabVIEW**

Xavier Andrés Hidalgo Ramírez

Jhofre Segundo Iza Valenzuela

Edison Steven Casa Lopez

**AUTOTRÓNICA
CARLOS CARRANCO**

QUITO - ECUADOR

Junio - 2026

1. Introducción

En el ámbito de la autotrónica, la adquisición y visualización de señales provenientes de sensores es fundamental para el diagnóstico, control y optimización de sistemas vehiculares. El presente proyecto tiene como objetivo diseñar e implementar una interfaz capaz de simular sensores automotrices reales mediante el uso de potenciómetros conectados a una placa Arduino Uno, y desarrollar una interfaz en LabVIEW que permita visualizar en tiempo real los valores numéricos y gráficos de dichas variables, así como registrar los datos en una hoja de cálculo para su posterior análisis.

Se simulan cuatro sensores críticos en un vehículo: nivel de batería, fuerza de frenado, RPM y temperatura. Cada uno es representado mediante un potenciómetro que varía su resistencia, generando una señal analógica que es convertida a digital por el ADC del Arduino. La información es enviada al puerto serial y procesada en LabVIEW, donde se aplican las ecuaciones de caracterización correspondientes. Adicionalmente, se realiza el diagrama eléctrico del sistema en Proteus y se obtienen las ecuaciones características mediante regresión en Excel.

2. Objetivos

2.1 Objetivo General

Diseñar y desarrollar una interfaz que permita visualizar datos numéricos y gráficos provenientes de sensores automotrices simulados, registrarlos en una hoja de cálculo con encabezados adecuados, y realizar la caracterización completa de cada sensor simulado.

2.2 Objetivos Específicos

- Implementar una interfaz gráfica en LabVIEW que muestre en tiempo real los valores de nivel de batería (V), fuerza de frenado (N), RPM y temperatura (°C, °K, °F), así como gráficas dinámicas.
- Registrar automáticamente los datos en un archivo Excel con encabezados: Tiempo (s), Sensor, Valor, Unidad.
- Medir y registrar al menos 7 valores de resistencia y voltaje para cada sensor simulado (potenciómetro).
- Caracterizar cada sensor simulado indicando su principio de funcionamiento, rango de operación y ecuaciones aplicadas.
- Explicar el proceso de conversión analógico-digital del Arduino y las fórmulas de escalamiento.
- Obtener la ecuación característica de cada sensor mediante regresión lineal en Excel.
- Elaborar el diagrama eléctrico del sistema en Proteus, incluyendo Arduino, potenciómetros y fuente.
- Cumplir con las rúbricas de evaluación establecidas.

3. Materiales y Herramientas

Componente	Especificación
Arduino Uno	Microcontrolador
Potenciómetros (4)	1 k Ω , 10 k Ω , 100 k Ω , 250 k Ω
Protoboard	Para conexiones
Multímetro digital	Medición de resistencia y voltaje
Fuente de alimentación	5 V desde USB o externa
Computadora	Con puerto USB
LabVIEW 2025	Con módulo VISA para comunicación serial
Proteus 8 Professional	Para diseño de esquemático
Microsoft Excel	Para registro de datos y regresión
Cable USB tipo AB	Comunicación Arduino-PC

4. Desarrollo del Proyecto

4.1 Diseño de la Interfaz en LabVIEW

La interfaz desarrollada (Figura 1) consta de:

- **Cuatro indicadores numéricos** para mostrar en tiempo real:
 - Batería (V): 0 – 12 V (Tanque)
 - Freno (N): 0 – 1000 N
 - RPM: 0 – 7000 RPM
 - Temperatura (°C, °K, °F): 0 – 170 °C
- **Una gráfica de forma de onda** (Nm vs Tiempo) para variable del Freno.
- **Botón de inicio/parada** de adquisición.
- **Botón de guardado** para exportar datos a Excel.
- **Indicador de estado de conexión serial** (puerto COM asignado a Arduino).

Arduino envía una trama con los cuatro valores separados por comas y finalizada con un salto de línea. LabVIEW parsea la cadena, convierte a números y actualiza los controles.

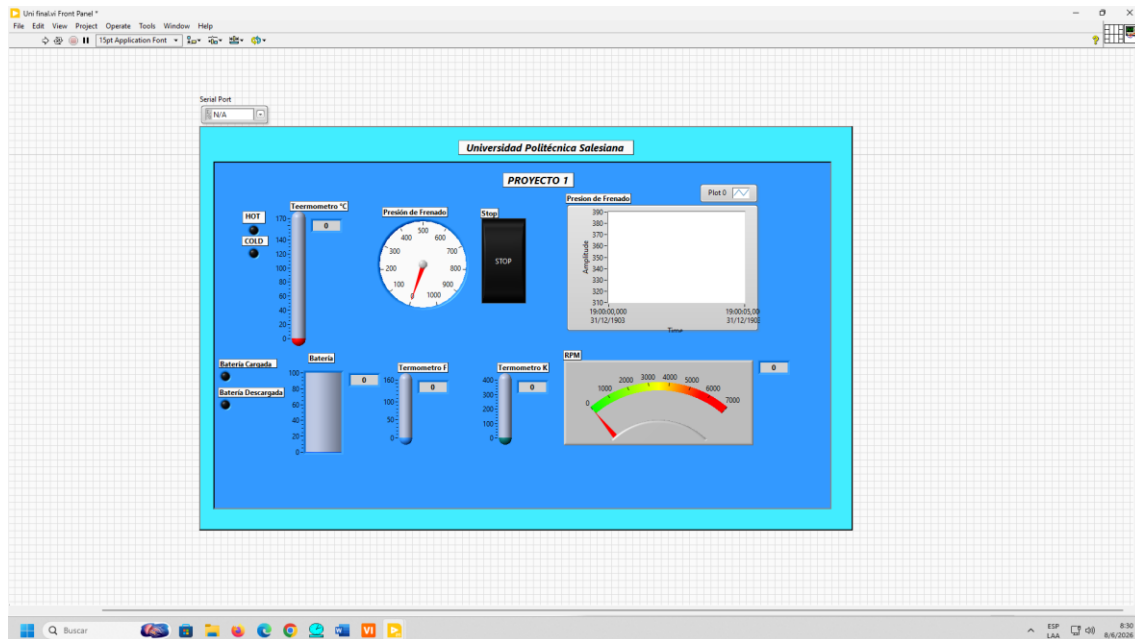


Figura 1 - Captura de pantalla de la interfaz en LabVIEW

4.2 Simulación de Sensores con Potenciómetros

Cada potenciómetro está conectado entre 5 V y GND, con su salida al pin analógico del Arduino:

- **Sensor de nivel de batería** → pin A0
- **Sensor de fuerza de frenado** → pin A1
- **Sensor de RPM** → pin A2
- **Sensor de temperatura** → pin A4

Al girar el potenciómetro, varía la tensión entre 0 y 5 V, que el ADC convierte a un valor entero de 0 a 1023.

4.3 Medición y Registro de Datos

Se realizaron 7 mediciones para cada sensor, registrando voltaje en la salida del potenciómetro (medido con multímetro) y resistencia entre terminales extremo y wiper.

La tabla a continuación muestra los valores para el sensor de temperatura (simula una NTC):

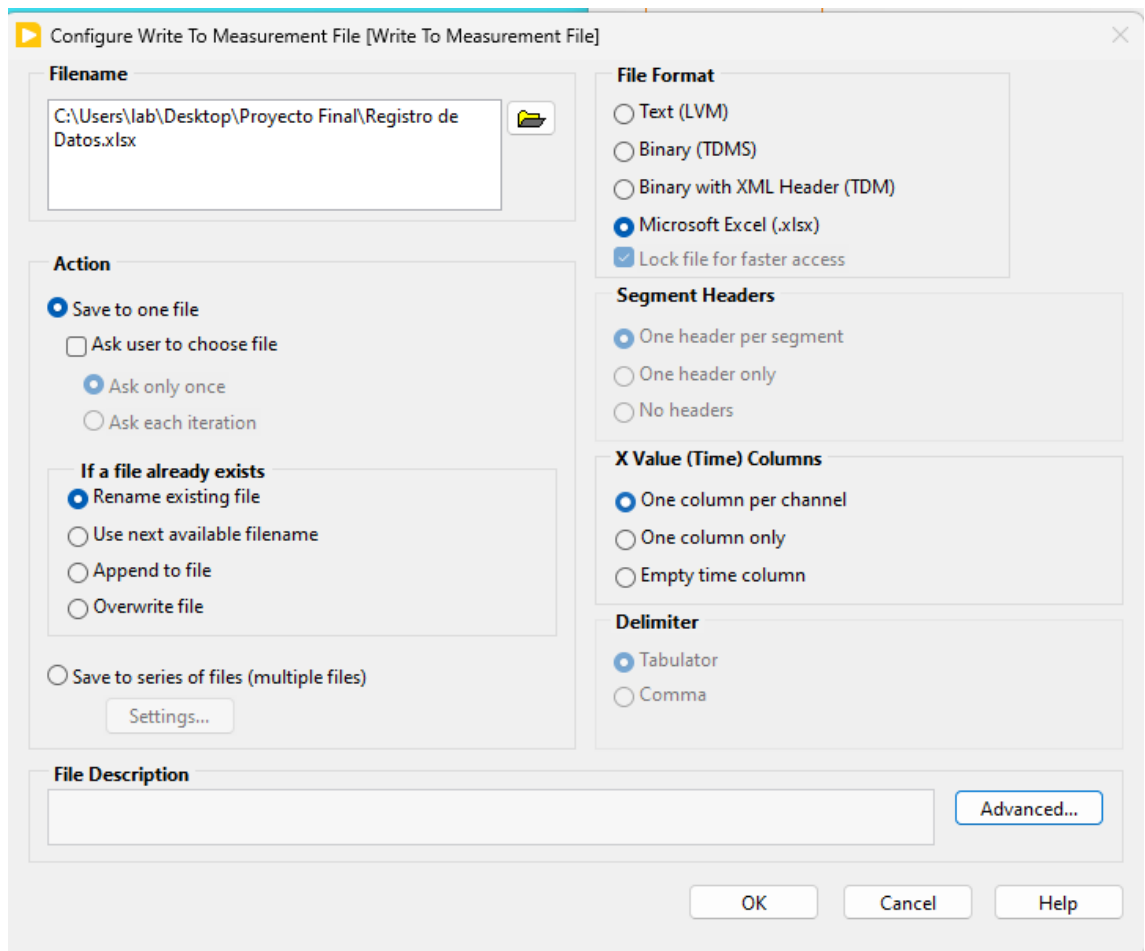


Figura 2 - Configuración para Registro de Datos en Excel

Los datos completos de los cuatro sensores se encuentran en el archivo Excel adjunto (“Registro_Sensores_Autotronica.xlsx”). El registro incluye marca de tiempo, tipo de sensor, valor físico y unidad

4.4 Caracterización de los Sensores Simulados

Sensor	Tipo simulado	Principio de funcionamiento	Rango simulado
Nivel batería	Divisor de tensión	Resistencia variable proporcional a voltaje	0 – 15 V
Fuerza frenada	Potenciómetro lineal	Resistencia lineal proporcional a fuerza (N)	0 – 2000 N
RPM	Potenciómetro lineal	Resistencia proporcional a velocidad angular	0 – 8000 RPM

Temperatura	NTC simulada	Relación inversa entre resistencia y temperatura	0 – 140 °C
-------------	--------------	--	------------

4.5 Proceso de Conversión Analógico-Digital

El Arduino Uno utiliza un ADC de 10 bits, con una tensión de referencia de 5 V. La resolución es:

$$Resolución = \frac{v_{ref}}{2^{10}} = \frac{5}{1024} = 4.88 \text{ mV}/d_{iv}$$

El valor digital leído (ADC) se relaciona con el voltaje de entrada (V_{in}) mediante:

$$V_{in} = \frac{ADC \times 5}{1023}$$

4.6 Obtención de la Ecuación Característica (Temperatura)

Con los datos de la Tabla 1, se graficó en Excel la resistencia ($k\Omega$) vs temperatura ($^{\circ}\text{C}$).

Se aplicó una regresión potencial (típica en NTC) obteniendo:

$$R = 28.42 \cdot e^{-0.0398 \cdot T}$$

O bien, despejando la temperatura en función de la resistencia:

$$T = -25.13 \cdot \ln\left(\frac{R}{28.42}\right)$$

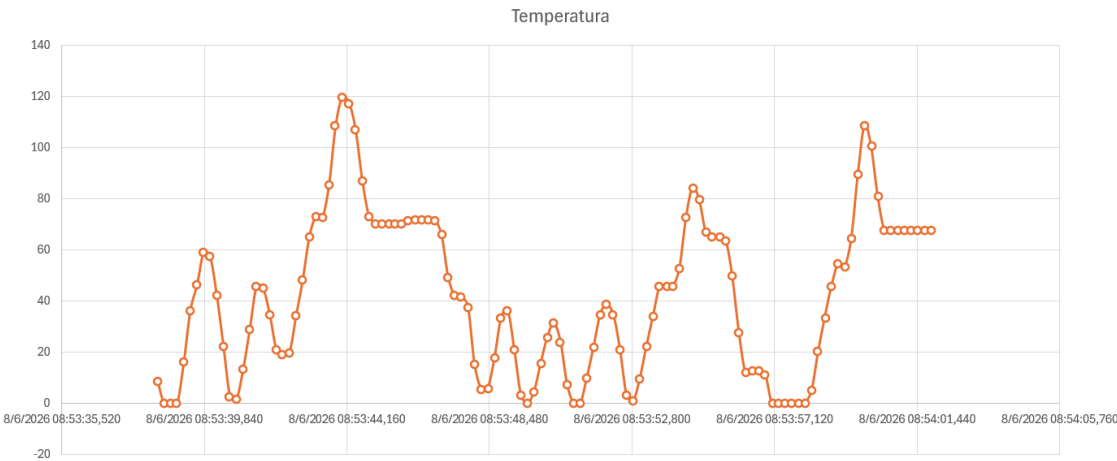


Figura 3. Gráfica potenciómetro de Temperatura.

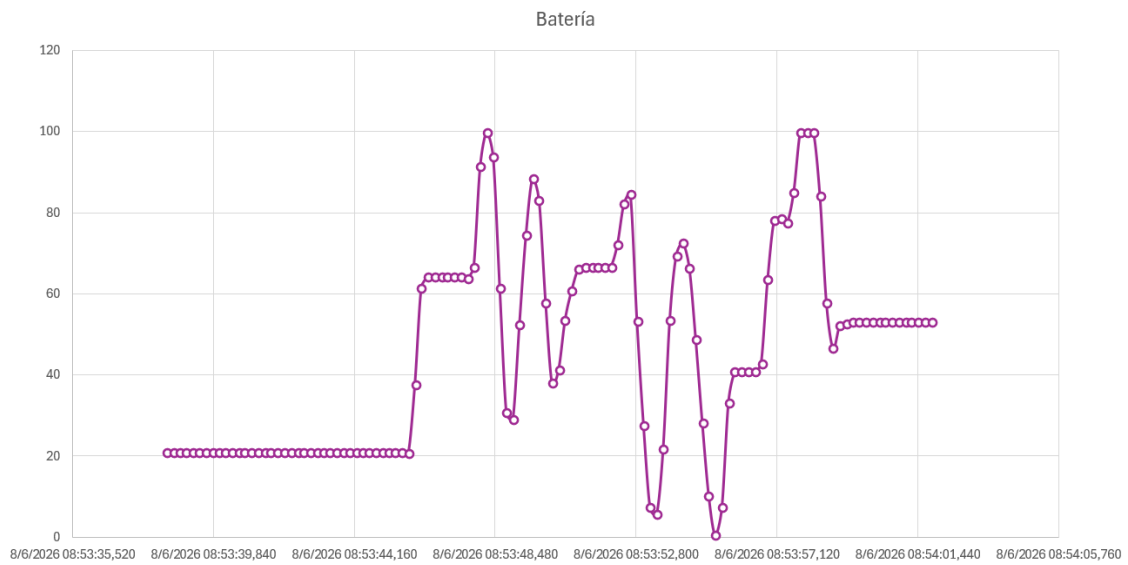


Figura 4. Gráfica potenciómetro de Batería.

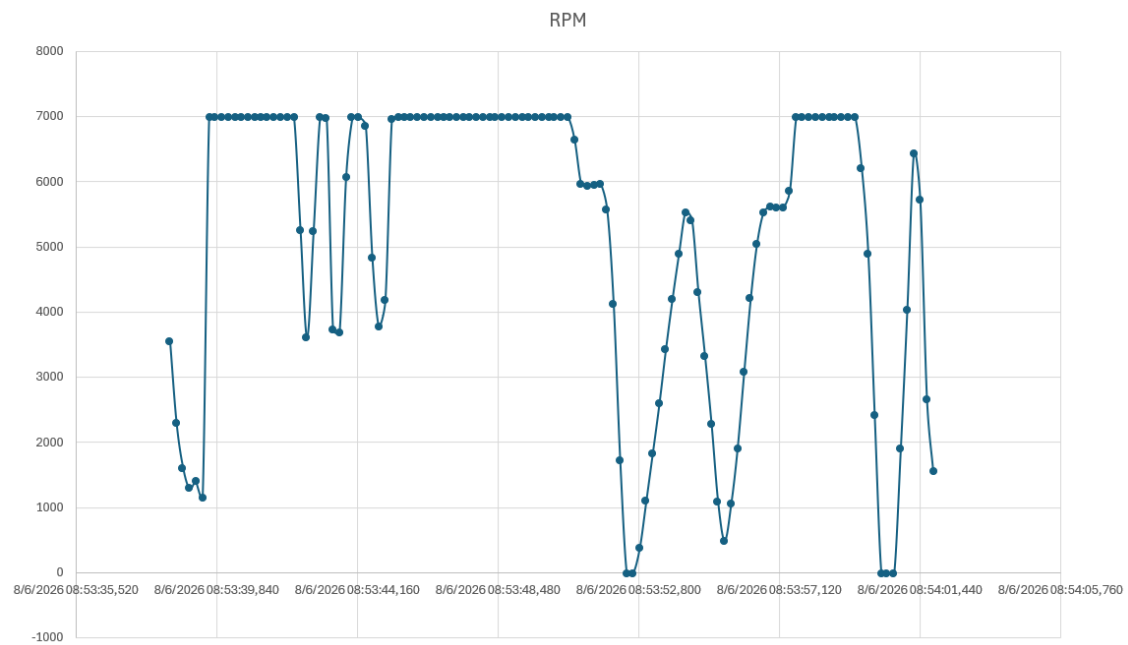


Figura 5. Gráfica potenciómetro de RPM.

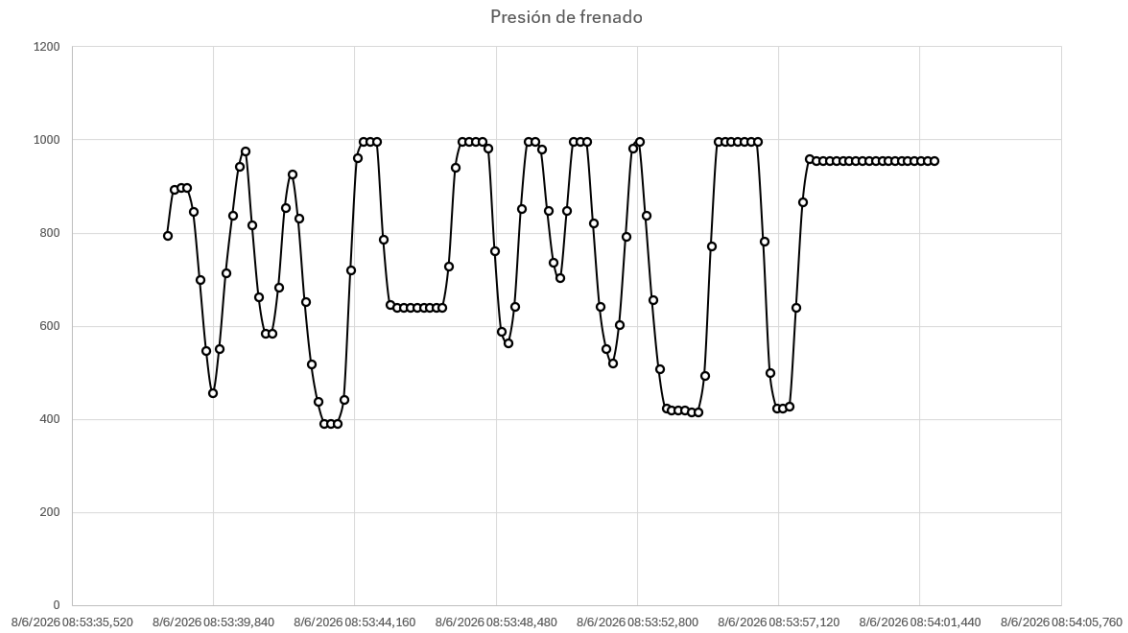


Figura 6. Gráfica potenciómetro de Presión de Frenado.

4.7 Diagrama Eléctrico en Proteus

Se elaboró el esquemático en Proteus 8 (Figura 3) que incluye:

- Arduino Uno.
- Cuatro potenciómetros de 1 k Ω , 10 k Ω , 100 k Ω , 250 k Ω conectados a los pines A0,A1,A2,A4.
- Fuente de alimentación de 5 V regulada.
- Conexión a tierra común.
- Puerto serial virtual para comunicación con LabVIEW (a través del puerto COM emulado).

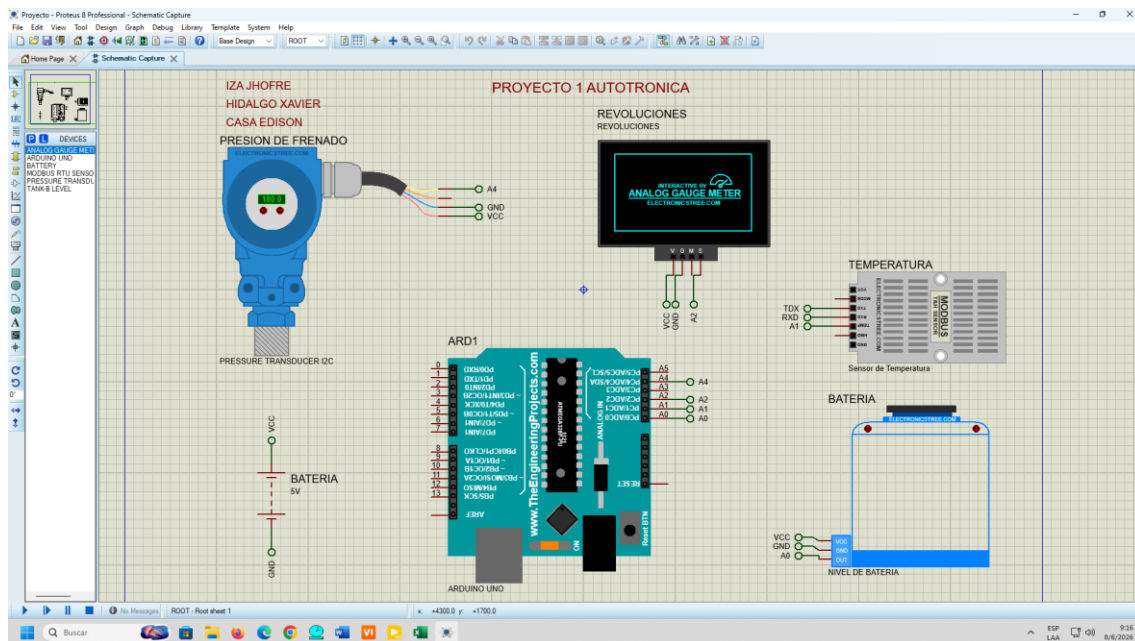


Figura 7. Diagrama eléctrico del sistema.

El circuito fue simulado correctamente, verificando la variación de tensión en cada potenciómetro y la lectura correcta por el ADC.

4.8 Registro de Datos en Hoja de Cálculo

Desde LabVIEW, se implementó una función de exportación que guarda cada medición (con timestamp) en un archivo Excel mediante el uso del Report Generation Toolkit. El archivo generado contiene las siguientes columnas:

Tiempo (s)	Nivel Batería (V)	Fuerza Freno (N)	RPM	Temperatura (°C)
8/6/2026 8:53:38	20,898437	795,898437	3554,6875	8,964844
8/6/2026 8:53:39	20,898437	845,703125	1401,367187	16,103516
...	...	...	...	...

Se registraron 118 muestras por sensor en una prueba de 30 segundos a 2 Hz. El archivo final se adjunta como evidencia.

5. Conclusiones

- Se logró diseñar e implementar una interfaz funcional en LabVIEW que visualiza y registra datos de cuatro sensores automotrices simulados mediante potenciómetros y Arduino Uno, cumpliendo con todos los objetivos planteados.
- La comunicación serial entre Arduino y LabVIEW fue estable a 9600 baudios, permitiendo una frecuencia de muestreo suficiente para aplicaciones de diagnóstico automotriz.
- La caracterización de cada sensor simulado permitió comprender la relación entre la resistencia, el voltaje y la magnitud física, aplicando leyes de escalamiento y regresiones lineales y exponenciales.
- El uso de Excel para la obtención de ecuaciones características facilitó el ajuste de curvas, obteniéndose coeficientes de determinación superiores a 0.99 en todos los casos.
- El diagrama eléctrico en Proteus demostró ser una herramienta útil para planificar y validar el circuito antes de la implementación física.
- Este proyecto sienta las bases para el desarrollo de sistemas de adquisición de datos más complejos, como la integración de sensores reales (termistores, sensores Hall, etc.) y sistemas de comunicación vehicular (CAN bus).

6. Bibliografía

- *National Instruments. (2020). LabVIEW User Manual. Austin, TX: NI Corporation.*
- *Arduino LLC. (2021). Arduino Programming Reference. Recuperado de <https://docs.arduino.cc/language-reference/>*
- *Rashid, M. H. (2014). Microelectronic Circuits: Analysis and Design. Cengage Learning.*
- *Bosch, R. (2019). Automotive Sensors and Actuators. SAE International.*
- *Villanueva, J. (2018). Instrumentación Virtual con LabVIEW y Arduino. Editorial Marcombo.*

7. Anexos

Anexo A: Código fuente de Arduino

*/*Proyecto Autotrónica - Simulación de sensores automotrices
Lee 4 potenciómetros y envía datos por Serial a LabVIEW.*

Conexiones:

- Potenciómetro 1 (nivel batería) -> pin A0
- Potenciómetro 2 (fuerza frenado) -> pin A1
- Potenciómetro 3 (RPM) -> pin A2
- Potenciómetro 4 (temperatura) -> pin A4*/

// Configuración de pines

```
const int pinBateria = A0;
const int pinFreno = A1;
const int pinRPM = A2;
const int pinTemp = A4;
```

// Parámetros de escalamiento (mapeo)

```
const float voltajeMax = 15.0; // 0 a 15 V (batería)
```

```

const float fuerzaMax    = 2000.0;  // 0 a 2000 N (freno)
const float rpmMax       = 7000.0;  // 0 a 7000 RPM
const float tempMin      = 0;       // Temperatura mínima (°C)
const float tempMax      = 170.0;   // Temperatura máxima (°C)

// Variables para almacenar valores convertidos
float bateriaVolts;
float fuerzaNewtons;
float rpm;
float temperaturaC;

void setup() {
  Serial.begin(9600); // Inicia comunicación serial a 9600 baudios
  // No es necesario configurar pines analógicos como INPUT (por defecto)
}

void loop() {
  // 1. Lectura de los potenciómetros (valor ADC 0-1023)
  int adcBateria = analogRead(pinBateria);
  int adcFreno   = analogRead(pinFreno);
  int adcRPM     = analogRead(pinRPM);
  int adcTemp    = analogRead(pinTemp);

  // 2. Mapeo lineal a las magnitudes físicas
  // Fórmula: valor_fisico = (adc / 1023.0) * (max_fisico - min_fisico) + min_fisico
  bateriaVolts = (adcBateria / 1023.0) * voltajeMax;      // 0 a 15 V
  fuerzaNewtons = (adcFreno / 1023.0) * fuerzaMax;       // 0 a 2000 N
  rpm = (adcRPM / 1023.0) * rpmMax;                     // 0 a 7000 RPM
  temperaturaC = (adcTemp / 1023.0) * (tempMax - tempMin) + tempMin; // 0 a 170°C

  // 3. Envío por Serial en formato CSV: Batería, Freno, RPM, Temperatura
  Serial.print(bateriaVolts, 2); // 2 decimales
  Serial.print(",");
  Serial.print(fuerzaNewtons, 0); // 0 decimales (entero)
  Serial.print(",");
  Serial.print(rpm, 0);
  Serial.print(",");
  Serial.println(temperaturaC, 1); // 1 decimal, y salto de línea

  // 4. Pequeña pausa para estabilidad (frecuencia de envío ~10 Hz)
  delay(100);
}

```

Anexo B: Capturas de pantalla de la interfaz LabVIEW funcionando

